

Sistemas de Comunicación y Seguridad Informática

Semana 7 13 de octubre de 2026: DevSecOps y Seguridad Cloud

Integrando la seguridad en el ciclo de vida del desarrollo.

Docente: Mg. Gabriel Guerrero Arellano

¿Qué haremos en estas 2 horas?

Ubicación en el currículo: Momento 2: Vulnerabilidad y Desarrollo.

Conceptos Clave: DevSecOps y Seguridad Cloud.

Temática específica: CI/CD seguro, análisis de código estático y arquitectura en la nube.

Herramientas: GitHub Actions, Docker, Trivy.

Laboratorio Práctico: Despliegue de un pipeline automatizado con escaneo de seguridad.

Evidencia: Entrega de resultados del escaneo.

¿Dónde estamos?

Estamos en el **Momento 2: Vulnerabilidad y Desarrollo**.

1. Ya pasamos por la Semana 5: Explotación de Vulnerabilidades.

2. Avanzamos por la Semana 6: Desarrollo Seguro y Apps.

3. Hoy daremos el salto a la automatización y la arquitectura en la nube.

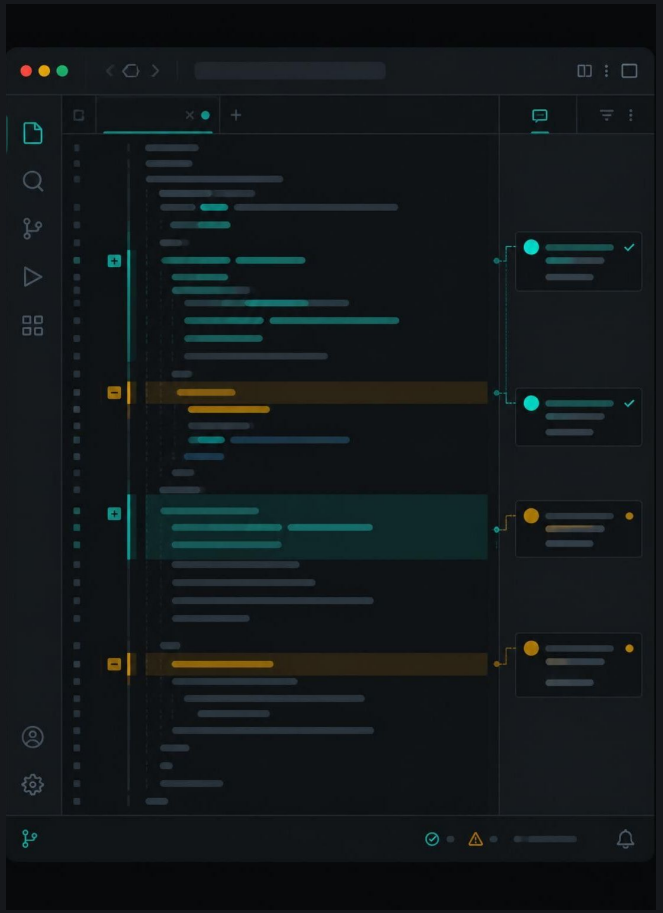


La Evolución: De DevOps a DevSecOps

El Problema: La seguridad tradicional era un "cuello de botella" al final del desarrollo.

La Solución (DevSecOps): Filosofía que integra la seguridad como una responsabilidad compartida desde el inicio.

Shift-Left Security: "Mover hacia la izquierda" significa aplicar pruebas de seguridad en las etapas más tempranas (diseño y código), no solo en producción.





Seguridad y Arquitectura en la Nube

La arquitectura en la nube cambia el paradigma de seguridad perimetral tradicional.

Modelo de Responsabilidad Compartida: El proveedor protege *la* nube, pero tú proteges lo que está *en* la nube (tus datos, tus apps, tu código).

La infraestructura como código (IaC) requiere ser escaneada al igual que el software.

Integración y Despliegue Continuo (CI/CD Seguro)

1

CI (Continuous Integration): Fusión constante de código. Aquí aplicamos pruebas automáticas.

2

El toque "Sec": Un CI/CD seguro detiene automáticamente el paso a producción si se detecta una vulnerabilidad crítica.

3

CD (Continuous Deployment): Despliegue automático a producción.

Análisis de Código Estático (SAST)



¿Qué es? Revisión de código fuente, bytecode o binarios sin ejecutar el programa.



Objetivo: Detectar inyecciones SQL, credenciales expuestas (hardcoded) o librerías vulnerables antes de compilar.



Ventaja: Feedback inmediato al desarrollador.

Anatomía de un Ataque por Inyección SQL y Desarrollo Seguro

1. EL ESCENARIO NORMAL (INPUT INOFENSIVO)





Recepción y concatenación directe sin filtrado.

El código backend de la tienda recibe lo que el usuarlo busca y lo inserta directamente en la consulta a la base de datos sin filtrario.



PRODUCTOS

CONSULTA GENERADA PARA ENTRADA INOFENSIVA

```
SELECT *
FROM productos
WHERE nombre = 'botas';
```

productos		
#	nombre	categoría
1	botas	--

2. LA ENTRADA MALICIOSA (EL ATAQUE PASO A PASO)



Buscar Producto (Malicioso)

botas'; DRDP TABLE usuarios; --



Inyección de testo estructurado en la barra de búsqueda.

El atacante escribe el texto exacto para manipular la consulta.



PRODUCTOS

CONSULTA ENSAMBLADA VULNERABLE

```
SQL
SELECT *
FROM productos
WHERE nombre = 'botas';

DROP TABLE usuarios; --
--';
```

3. DESCOMPOSICIÓN TÉCNICA DE LA EJECUCIÓN CATASTRÓFICA



EJECUCIÓN CATASTRÓFICA



BÚSQUEDA INICIAL LEGÍTIMA

El motor lee `SELECT * FROM productos WHERE nombre = `botaa¨ y realiza la búsqueda inofensiva.

CARÁCTER DE TERMINACIÓN (;)

El punto y coma le indica a la base de datos que ha terminade un comando y comienza una nueva instrucción.





COMANDO DESTRUCTIVO

Se ojecuta `DROP TABLE usuarios;`, horrando por completo la tabla con los registros de clientas y sus contraseñas.

ANULACIÓN DE SINTAXIS CON GUIONES (--)



IMPACTO INMEDIATO EN LA BASE DE DATOS

Pérdida total de la base de datos en un milisegundo provocada desde una simple barra de busqueda.

4. LA REGLA DE ORO DEL DESARROLLO SEGURO Y PREVENCIÓN CON TRIVY



LA REGLA DE ORO DEL DESARROLLO SEGURO

Jamás confiar en el input del usuario y parametrizar siempre las consultas mediante Prepared Statements.

```
SELECT *
FROM productos
WHERE nombre = ?;
```

PARAMETRIZACIÓN: El input se treta como dato, no como codigo.





PREVENCIÓN CON TRIVY Y ESCANERES DE SEGURIDAD.

Permiten la identificación automática de vulnerabilidades, dependencias inseguras y malas configuraciones de seguridad en el código y contenedores antes de llegar a producción.

Gemini Notebook

El Ecosistema de la Semana 7

01

Codificamos (Desarrollo).

02

Enviamos al repositorio (Git).

03

Se dispara la automatización (CI/CD seguro).

04

Se ejecuta el análisis de código estático.

05

Si es seguro, se despliega en la arquitectura en la nube.
Si no, se bloquea.

Herramientas a Utilizar

Pasemos de la teoría a la práctica. ¿Qué arsenal utilizaremos hoy?

EL GUARDIÁN DEL PIPELINE

Aqua Trivy: Seguridad a la velocidad de la Nube



¿Qué es?

Un escáner de vulnerabilidades (SAST/SCA) de código abierto, diseñado específicamente para integrarse en CI/CD.



Ultra Rápido (Stateless):

A diferencia de escáneres antiguos, no requiere instalar bases de datos pesadas locales; descarga las firmas de seguridad en segundos y analiza al instante.



Escaneo Integral:

Es multipropósito. Analiza tu código fuente, las librerías de terceros (ej. npm), la imagen del sistema operativo (Docker) y tu infraestructura en la nube.



El Rol de Hoy:

Es el "portero" automatizado en nuestro GitHub Actions. Su trabajo es detectar los CVEs (Vulnerabilidades Comunes) y emitir un código de error (`exit-code: 1`) para bloquear inmediatamente el paso a producción si el código es peligroso.

El motor de nuestro CI/CD

- **GitHub:** Alojamiento de nuestro repositorio de código.
- **GitHub Actions:** Plataforma de CI/CD nativa.
- Nos permite crear flujos de trabajo (workflows) definidos en archivos `.yml` que se ejecutan automáticamente ante un evento (ej. un "push").





Estándar de Arquitectura Cloud

- Contenedores ligeros y portables.
- Permite empaquetar la aplicación con todas sus dependencias.
- En DevSecOps, el archivo `Dockerfile` también debe ser auditado, ya que una imagen base vulnerable compromete toda la aplicación en la nube.

Laboratorio Práctico: CI/CD Seguro en Acción

Objetivo: Crear un pipeline que construya una imagen de Docker y pase por un análisis de seguridad con Trivy. Si hay vulnerabilidades críticas, el pipeline debe fallar intencionalmente.

Fase 0: Validación de Entorno

Atención a todos: No avanzamos al Paso 1 hasta que la terminal de todos devuelva un "OK" en estos 3 comandos.

01

Validar Git: `git --version`

02

Validar Docker: `docker --version`

03

Validar Cuenta: Iniciar sesión en github.com en sus navegadores.

Troubleshooting rápido: Si Docker no arranca, verifiquen que Docker Desktop esté abierto.

Fase de Validación: Preparando el Entorno Paso a Paso

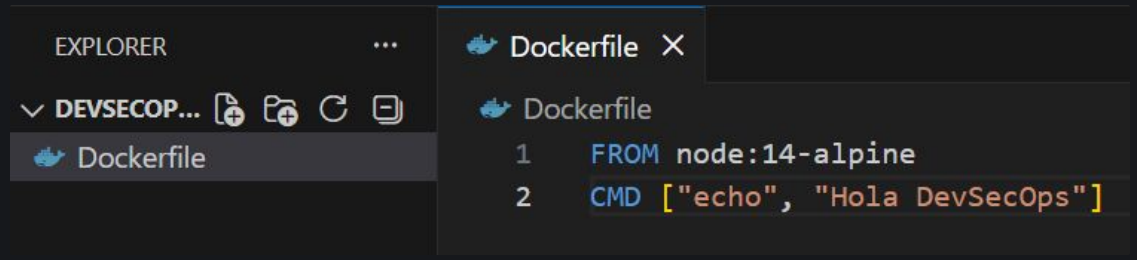
Sigue estos comandos en tu terminal uno a uno para evitar errores de sincronización.

1. **Crear carpeta local y entrar en ella:**

```
mkdir devsecops-lab
cd devsecops-lab
```

2. **Crear el archivo Dockerfile en la raíz con este contenido (vulnerable intencionalmente):**

```
FROM node:14-alpine
CMD ["echo", "Hola DevSecOps"]
```



3. **Inicializar Git localmente:**

```
git init
git add Dockerfile
git commit -m "Commit inicial: Dockerfile"
```

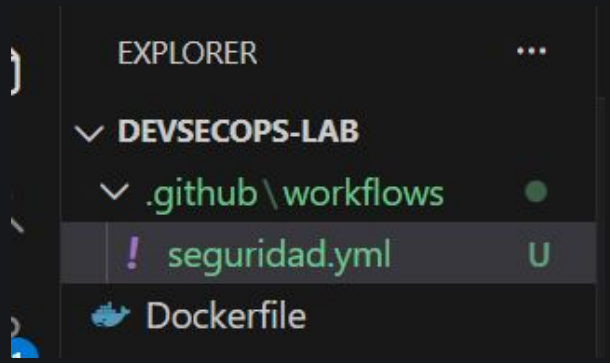
4. **Conectar con GitHub (Sincronización):** Ve a GitHub.com y crea un repositorio *vacío* (sin README ni .gitignore) llamado `devsecops-lab`. Copia los comandos que te da GitHub, que se ven así, ejecuta en la terminal de vs code con ctrl+ñ:

```
git remote add origin
https://github.com/GaboXbox/devsecops-lab.git

git branch -M main
git push -u origin main
```

Paso 2: Creando el Workflow de CI/CD

En el repositorio local, crear la ruta: `.github/workflows/seguridad.yml` el arbol, quedara así:



Dentro del archivo llamado seguridad.yml pegar el siguiente codigo:

en este momento Definiremos los "Jobs":

1. Construir (Build) la imagen de Docker.
2. Escanear la imagen.

Paso 3: El código de GitHub Actions

```yaml
name: DevSecOps Pipeline
on: [push]
jobs:
  build-and-scan:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - name: Construir Imagen
        run: docker build -t mi-app-insegura:latest .
      - name: Escaneo Trivy
        uses: aquasecurity/trivy-action@master
        with:
          image-ref: 'mi-app-insegura:latest'
          format: 'table'
          exit-code: '1'
          severity: 'CRITICAL,HIGH'
```

Paso 4: ¡Push y Análisis!

Lanzamos el código a la nube, desde terminal de comandos de vs code:

```
git add .
git commit -m "Implementando DevSecOps"
git push origin main
```

Vamos a la pestaña "Actions" en nuestro repositorio de GitHub y observamos la magia.

¿Qué acaba de pasar?



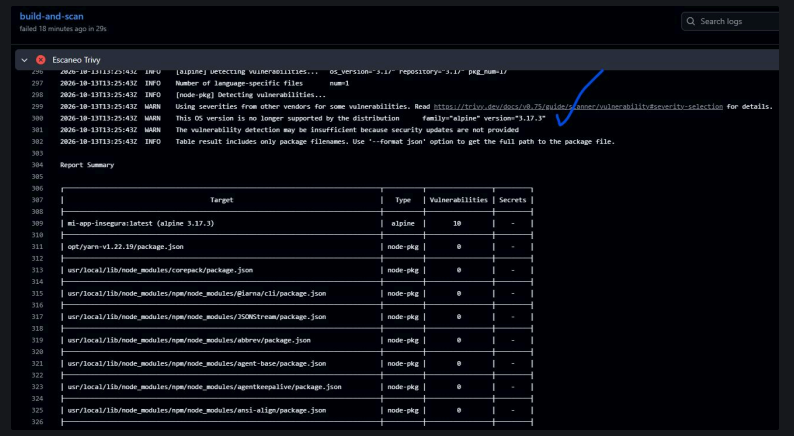
- El pipeline **FALLÓ** (X roja en GitHub Actions). (**EXCELENTE)**
- **GitHub Actions se activó:** Al hacer el *push*, GitHub detectó tu archivo `seguridad.yml` y arrancó el proceso (job) llamado `build-and-scan`.
- **Construyó la imagen:** Logró construir la imagen de Docker usando el archivo `Dockerfile`.
- **Trivy entró en acción y bloqueó todo:** El escáner analizó esa imagen y descubrió que `node:14-alpine` está lleno de vulnerabilidades. Como en nuestro código le dijimos que si encontraba vulnerabilidades críticas o altas debía arrojar un error (`exit-code: '1'`), Trivy detuvo el proceso inmediatamente.
- **Protección exitosa:** El pipeline falló intencionalmente para evitar que ese código inseguro pasara a la siguiente etapa o a producción. ¡Eso es DevSecOps puro!



Anatomía del Escaneo: ¿Qué nos dice Trivy?



1. **Navegando al error:** En GitHub, ve a la pestaña "Actions", haz clic en el workflow fallido y luego en el trabajo **"build-and-scan"**.
2. **Abriendo los Logs:** Despliega la flecha que dice "Escaneo Trivy" para ver la salida de la consola.
3. **Advertencia Crítica (WARN):** El sistema alerta que estamos usando un sistema operativo base obsoleto (`alpine 3.17.3` ya no recibe parches de seguridad) LINE 300
4. **Tabla de Resumen (Report Summary):** Trivy muestra el conteo total. En este caso, detectó **10 vulnerabilidades** a nivel del sistema base.
5. **El Bloqueo (`exit-code: 1`):** Al encontrar estas fallas, Trivy activó la orden de nuestro archivo `.yml` de abortar la misión, evitando que el código inseguro avance.



Evidencia de Aprendizaje de la Semana 7

Para validar la asistencia y participación de hoy:

01

URL de su repositorio público en GitHub.

02

Captura de pantalla de la pestaña "Actions" mostrando el pipeline fallido.

03

Captura de pantalla de los logs de Trivy mostrando al menos un CVE (vulnerabilidad) detectado.

Conclusión y Próximos Pasos

- La seguridad no es un paso final, es un proceso continuo.
- Las herramientas automáticas son esenciales en la arquitectura en la nube.
- "Romper" el pipeline a tiempo salva a las empresas de brechas de seguridad millonarias.

- **Próxima clase:** Semana 8: Pentesting Estructurado.
- **Temática próxima:** Fases de reconocimiento, escaneo de puertos y análisis de vulnerabilidades.